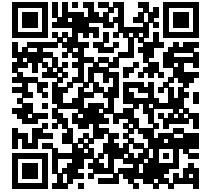


Digital Electronics — Course Notes

Printed version • Version: 26 September 2026

Where to write your answers: if you have access to a device, use the **online course notes** instead — you can type your answers into the answer panels and they are saved as you go. Otherwise, write your answers in your **jotter**, with the task and question number beside each answer.

Scan for the online
course notes

Section 1: Analogue and Digital Signals

Electronic circuits process **electrical signals**. A signal is a voltage that carries information — it tells the circuit when to switch on or off, or what value something is. There are two fundamentally different types of signal:

Analogue signals

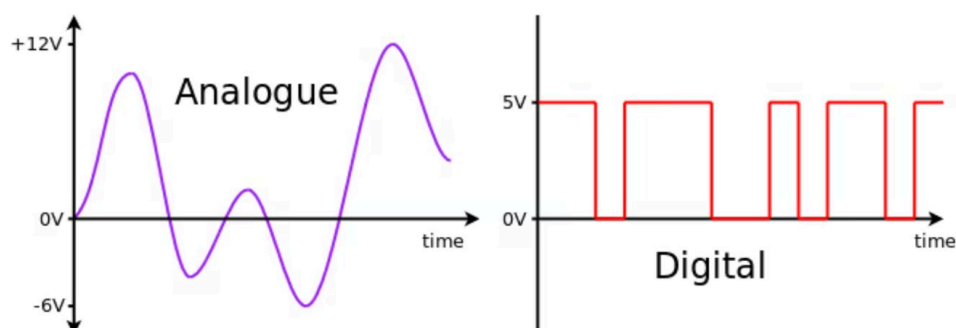
An **analogue signal** can be *any* voltage value between a minimum and maximum. It changes smoothly and continuously, like a dimmer switch or a volume control. A good example is the light level outside — it gradually gets brighter from dawn, reaches a peak at midday, and slowly fades to dark.

Digital signals

A **digital signal** can only be one of two values — **ON** or **OFF**. In electronics, these are represented as:

Logic 1 = ON = HIGH = approximately +5V Logic 0 = OFF = LOW = 0V

A simple switch is a digital input — it is either open (0) or closed (1). There is no in-between. All computers, smartphones and digital systems work entirely with 1s and 0s.



Task 1 — Analogue or digital?

For each of the following, state whether the signal is analogue or digital and explain your reasoning.

- a) The output of a thermistor (a component that changes resistance depending on temperature) as room temperature gradually rises
- b) The output of a push-button switch
- c) The output of a microphone picking up speech
- d) The value stored in a computer's memory

Section 2: Logic Gates

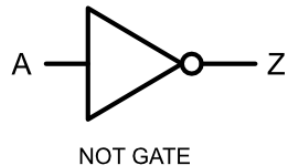
A **logic gate** is an electronic component that takes one or more digital inputs and produces a single digital output. The output depends entirely on the combination of inputs — and that relationship is described precisely using a **truth table** and a **Boolean expression**.

Logic gates are the building blocks of all digital systems — from simple alarm circuits to the billions of transistors inside a modern CPU (Central Processing Unit — the "brains" of a computer, which executes instructions and processes information).

For N5 Engineering Science you must know: NOT, AND and OR gates — their symbols, truth tables and Boolean expressions.

The NOT gate (Inverter)

The NOT gate has **one input** and **one output**. It simply **inverts** the input — if the input is 1, the output is 0, and vice versa. The small circle (bubble) on the output of the symbol represents inversion.



TRUTH TABLE

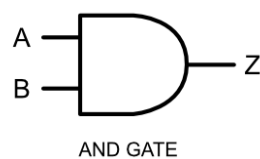
Input A	Output Z
0	1
1	0

A **truth table** lists every possible combination of inputs a gate can have, and shows the output that results for each one. This lets us describe exactly how a gate behaves, with no ambiguity — once you know a gate's truth table, you know everything about how it works.

Real-world example: A NOT gate is needed in a burglar alarm when a light beam is used. Light shines onto a sensor which gives a logic 1 output (beam intact = safe). When a burglar breaks the beam, the sensor output is turned off (logic 0). So a NOT gate is used to invert the signal. This enables a logic 1 signal to trigger an alarm.

The AND gate

The AND gate has **two (or more) inputs** and one output. The output is only logic 1 when **ALL inputs are logic 1** simultaneously. If any input is 0, the output is 0.



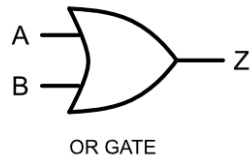
TRUTH TABLE

Input A	Input B	Output Z
0	0	0
0	1	0
1	0	0
1	1	1

Real-world example: A safety press on a machine — both the operator's left hand button (A) AND right hand button (B) must be pressed before the machine operates. This prevents the operator from accidentally putting a hand under the press.

The OR gate

The OR gate has **two (or more) inputs** and one output. The output is logic 1 when **ANY input is logic 1**. The output is only 0 when all inputs are 0.



TRUTH TABLE

Input A	Input B	Output Z
0	0	0
0	1	1
1	0	1
1	1	1

Real-world example: The courtesy light in a car. Input A: the driver's door is opened. Input B: the passenger's door is opened. Output: the interior roof light turns on. This is useful because you need the light to come on so you can see inside the car, regardless of which door is used to get in.

AND and OR gates aren't limited to two inputs — they can have three, or even four inputs. The gate still works the same way: an AND gate's output is only 1 when **all** inputs are 1, and an OR gate's output is 1 when **any** input is 1.

Task 2 — Logic gates

1. Draw the symbol for a 2-input AND gate (A, B), then complete the truth table below.

Input A	Input B	Output Z
0	0	
0	1	
1	0	
1	1	

2. Draw the symbol and complete the truth table for each gate.

- a) A 2-input AND gate (A, B).
- b) A 2-input OR gate (A, B).

3. For each situation below, name the most appropriate gate and explain why.

- a) A car alarm sounds if the door is opened OR the window is broken.
- b) A cash machine will only dispense money if a valid card is detected AND the correct PIN is entered.
- c) A fridge light that goes off when a switch is closed by the door shutting.



NoNANDSense Simulator — explore logic gates

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Section 3: Combinational Logic Circuits

When two or more logic gates are connected together to perform a specific task, the result is called a **combinational logic circuit**. The output depends on the *combination* of all current inputs.

How many rows in a truth table?

The number of rows in a truth table doubles with each additional input:

Number of inputs	Number of rows
1	2
2	4
3	8
4	16

The inputs are always listed in binary counting order — start from all 0s and count up in binary to all 1s. This ensures every combination is covered exactly once.

Just as each column in a decimal number represents a power of 10 (units, tens, hundreds...), each column in a binary number represents a power of 2. Reading from the right: the 1s column, then the 2s column, then 4s, then 8s — each column worth double the one before it.

There's a standard pattern to how the columns fill in as you count: the right-most column alternates every row (0, 1, 0, 1...), the column to its left alternates every 2 rows, the column after that every 4 rows, and so on. Each column added to the left doubles the pattern length — 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024... These are the same numbers you'll recognise from phone, computer and flash drive storage sizes — it's all built on the same binary counting system.

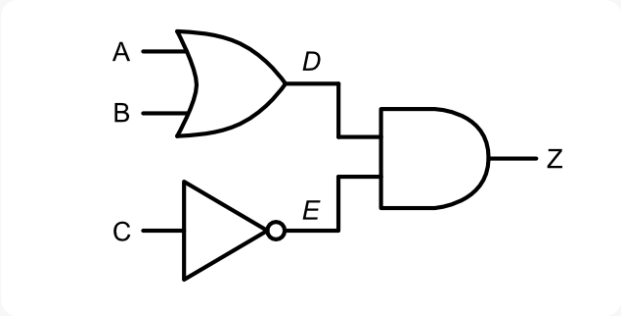
Here's decimal 0–15 built up from those column values:

Decimal	8s	4s	2s	1s	Binary
0	0	0	0	0	0000
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	0	1	1	0011
4	0	1	0	0	0100
5	0	1	0	1	0101
6	0	1	1	0	0110
7	0	1	1	1	0111
8	1	0	0	0	1000
9	1	0	0	1	1001
10	1	0	1	0	1010
11	1	0	1	1	1011
12	1	1	0	0	1100
13	1	1	0	1	1101
14	1	1	1	0	1110
15	1	1	1	1	1111

Method for completing a truth table from a circuit

Label the output of each intermediate gate with a letter (D, E, F etc.). Then fill in one column at a time, working left to right through the circuit.

WORKED EXAMPLE — COMPLETING A TRUTH TABLE STEP BY STEP



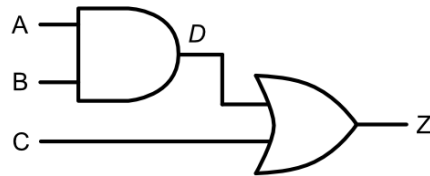
Circuit: A and B feed an OR gate (output D). C feeds a NOT gate (output E). D and E feed an AND gate (output Z).

- 1 Column D** (OR gate, inputs A and B): D = 1 whenever A OR B = 1.
- 2 Column E** (NOT gate, input C): E is the inverse of C.
- 3 Column Z** (AND gate, inputs D and E): Z = 1 only when D AND E are both 1.

A	B	C	D (A OR B)	E (NOT C)	Z (D AND E)
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0

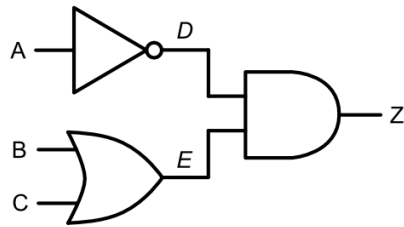
Task 3 — Complete the truth tables

For each circuit, complete the truth table by filling in the intermediate gate outputs and the final output. Check your answers using the NoNANDSense simulator.



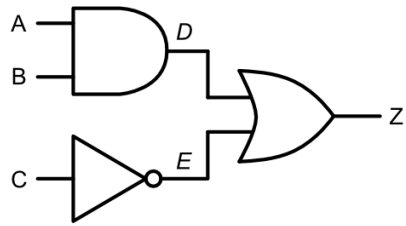
a) A and B feed an AND gate (output D). D and C feed an OR gate (output Z).

A	B	C	D (A AND B)	Z (D OR C)
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		



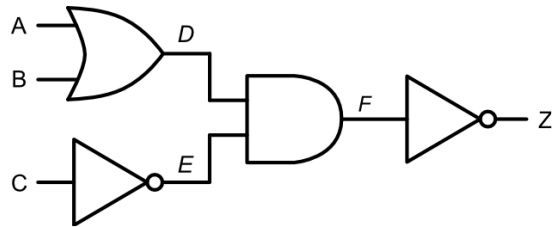
b) A feeds a NOT gate (output D). B and C feed an OR gate (output E). D and E feed an AND gate (output Z).

A	B	C	D (NOT A)	E (B OR C)	Z (D AND E)
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			



c) A and B feed an AND gate (output D). C feeds a NOT gate (output E). D and E feed an OR gate (output Z).

A	B	C	D (A AND B)	E (NOT C)	Z (D OR E)
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			



d) A and B feed an OR gate (output D). C feeds a NOT gate (output E). D and E feed an AND gate. The AND output feeds a NOT gate (output Z).

A	B	C	D (A OR B)	E (NOT C)	F (D AND E)	Z (NOT F)
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				



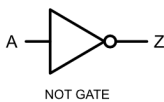
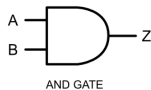
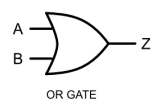
NoNANDSense Simulator — combinational circuits

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Section 4: Boolean Algebra

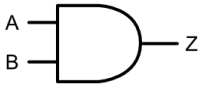
Boolean algebra was developed by the British mathematician George Boole in the 1850s. It gives us a precise mathematical language to describe the behaviour of logic circuits. Today it is used by every electronic engineer, computer scientist and software developer.

Boolean notation summary

Gate	Symbol	Meaning	Example
 NOT GATE	$\overline{A}$	NOT A	$Z = \overline{A}$ Z is ON when A is OFF
 AND GATE	$\cdot$	AND	$Z = A.B$ Z is ON when A AND B are ON
 OR GATE	$+$	OR	$Z = A+B$ Z is ON when A OR B (or both) are ON

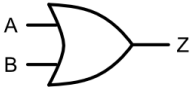
Single-gate examples

Before combining gates together, make sure you're comfortable writing the Boolean expression for a single gate on its own.


AND GATE

$$Z = A.B$$

"Z equals A AND B" — the dot means AND


OR GATE

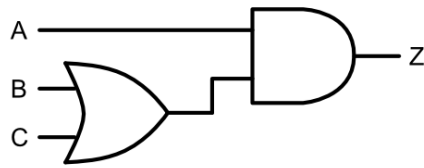
$$Z = A+B$$

"Z equals A OR B" — the + means OR

Writing a Boolean expression from a circuit

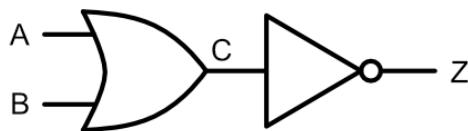
Work through the circuit left to right, writing an expression after each gate. Bracket sub-expressions where needed.

WORKED EXAMPLE — BOOLEAN EXPRESSION FROM CIRCUIT



- 1 Label the output of the OR gate (B,C): $B+C$
- 2 The AND gate takes A and (B+C) as inputs: $Z = A.(B+C)$
- 3 Complete expression: $Z = A.(B+C)$ — "Z is ON when A is ON AND at least one of B or C is ON"

WORKED EXAMPLE — A NOT GATE AT THE END OF A COMBINATION



- 1 Label the output of the OR gate (A, B): $A+B$
- 2 The NOT gate inverts the whole OR output — so the overline goes over the entire expression it's inverting, not just one letter: $Z = \overline{A+B}$
- 3 Complete expression: $Z = \overline{A+B}$ — "Z is ON only when neither A nor B is ON"

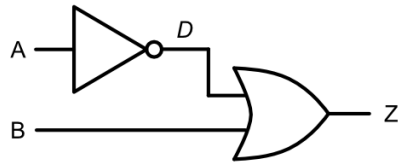
From circuit to truth table

Once you've labelled the intermediate points in a circuit, you can use those labels to build up a truth table one column at a time — just like the worked example below. Try it for the circuit where A and B feed an OR gate, and the OR output together with C feed an AND gate:

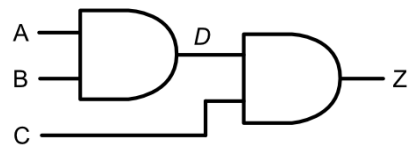
A	B	C	(A+B)	(A+B).C
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Task 4 — Boolean expressions from circuits

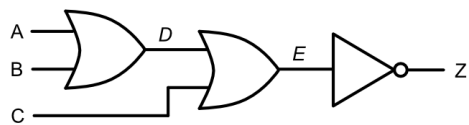
For each circuit below, complete a truth table and write the Boolean expression for Z.



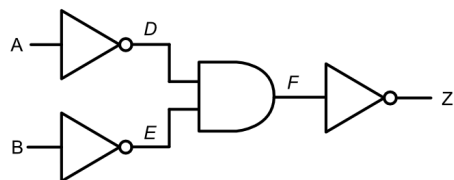
a) Complete the truth table and write the Boolean expression for Z.



b) Complete the truth table and write the Boolean expression for Z.



c) Complete the truth table and write the Boolean expression for Z.



d) Complete the truth table and write the Boolean expression for Z.

Section 5: Boolean Expressions from Truth Tables

Sometimes you will be given a truth table and asked to write the Boolean expression and draw the circuit. The method is straightforward — focus only on the rows where the output is **logic 1**.

Method

1. Look at the truth table and highlight every row where **Z = 1**
2. For each highlighted row, write an AND expression for that combination of inputs. If an input is 0, write its complement (with a bar): e.g. $\bar{A}$. If it is 1, write it normally: A.
3. Connect all the row expressions with OR (+) to get the full Boolean expression
4. Draw the logic circuit from the expression

WORKED EXAMPLE — BOOLEAN EXPRESSION FROM TRUTH TABLE

Truth table with 3 inputs A, B, C and output Z:

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

1 Row 3 (Z=1): A=0, B=1, C=0 → $\bar{A}.B.\bar{C}$

2 Row 5 (Z=1): A=1, B=0, C=0 → $A.\bar{B}.\bar{C}$

3 $Z = (\bar{A}.B.\bar{C}) + (A.\bar{B}.\bar{C})$

Task 5 — Boolean expressions from truth tables

For each truth table, write the Boolean expression for Z. Then build and verify the circuit using NoNANDSense.

a) Two inputs A and B:

A	B	Z
0	0	0
0	1	1
1	0	0
1	1	0

b) Two inputs A and B:

A	B	Z
0	0	1
0	1	0
1	0	1
1	1	0

c) Three inputs A, B, C:

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Section 6: Designing Circuits from a Specification

The most common engineering application of digital logic is designing a circuit to meet a given **written specification**. Read the specification carefully — the key words AND, OR and NOT tell you which gates are needed.

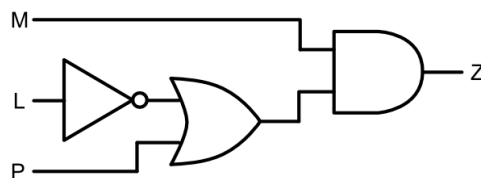
Method

1. Identify all the **inputs** from the specification
2. Identify the **output**
3. Read the specification and identify the logic: look for AND, OR, NOT
4. Write the **Boolean expression**
5. Draw the **logic circuit**
6. Draw up the **truth table** (all input combinations)
7. **Verify** using the simulator

WORKED EXAMPLE — MUSEUM BURGLAR ALARM

"A burglar alarm should sound if the master switch is on AND either a light beam is broken OR a pressure pad is triggered."

- 1 **Inputs:** Master switch (M), Light sensor (L), Pressure pad (P). **Output:** Alarm bell (B)
- 2 **Logic:** The light sensor triggers when the beam is *broken* — so it goes to 0 when a burglar passes. We need NOT L. The pressure pad goes to 1 when stood on.
- 3 **Boolean:** $B = M \cdot (\bar{L} + P)$
- 4 **Circuit:** L feeds a NOT gate. NOT-L and P feed an OR gate. The OR output and M feed an AND gate. The AND output is B.



M	L	P	$\bar{L}$	$\bar{L}+P$	B (alarm)
0	0	0	1	1	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	1	1

Task 6 — Design from specification

For each specification: identify inputs and outputs, write the Boolean expression, draw the circuit diagram and complete the truth table. Verify using NoNANDSense.

1. An arcade machine can only be played when the machine is switched on AND money has been inserted.
2. A central heating setup turns on heating when the primary switch is closed, the timer circuit is activated, and the manual bypass is disabled (logic 0).
3. A garage door opens automatically if the car headlights shine on a sensor (A) AND a sensor in the drive detects the car is present (B) — OR if a manual override switch (C) is pressed.
4. A jewellery cabinet alarm (Z) will sound if the cabinet door is opened (sensor A = 1) AND the sales assistant's override button is NOT pressed (button B = 1 when pressed).



NoNANDSense Simulator — design your circuits

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Section 7: Logic Gate Integrated Circuits (ICs)

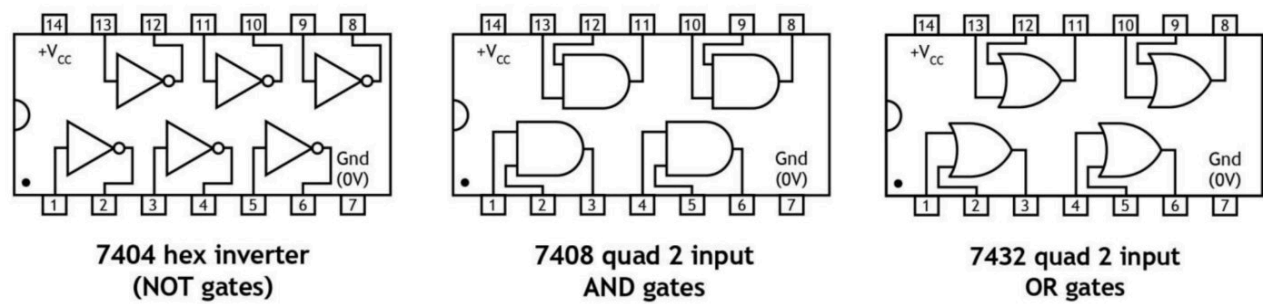
In practice, logic gates are not individual components — they are built into **integrated circuits (ICs)**, also called chips. Each IC contains several logic gates in a single small package.

The chips used in school are **TTL (Transistor-Transistor Logic)** chips, which require a stable **+5V supply**. The chip number always starts with **74**:

IC number	Contents	Description
7404	6 × NOT gates	Hex inverter — six independent NOT gates
7408	4 × 2-input AND gates	Quad 2-input AND
7432	4 × 2-input OR gates	Quad 2-input OR

Pin 14 connects to +5V (Vcc). Pin 7 connects to 0V (GND). These power connections must always be made or the IC will not work.

All three chips use the same 14-pin layout principle — Vcc on pin 14, GND on pin 7, with the gates arranged around the remaining pins:



You do **not** need to memorise the pin layout of these chips. The IC reference table above is an **assignment-specific data set** — it is not included in the SQA data booklet and won't be provided in an exam. What matters is understanding how many gates each IC contains, so you can choose the right chip(s) for a given circuit.

The NoNANDSense IC diagram feature

The NoNANDSense simulator includes a **7400 Series Circuit Diagram** tool. After building your logic circuit, this tool automatically selects the correct IC chips, groups gates into ICs, and draws the physical wiring diagram — ready for building on a breadboard.

Task 7 — Integrated circuits

1. A logic circuit uses two AND gates and one OR gate. Which ICs would you need? How many gates would be unused on each IC?
2. Using NoNANDSense, build the burglar alarm circuit from Section 6 and open the **7400 Series Circuit Diagram** tool.
 - a) List the ICs required to build this circuit.
 - b) Create a short wiring list for the first logic gate (e.g. Sensor A connects to Pin 1, Sensor B connects to Pin 2, Pin 3 connects to...).
3. Why must pin 14 be connected to +5V and pin 7 to 0V on every TTL IC?

Section 8: Real-World Applications of Logic Gates

Logic gates are found in almost every electronic system. Understanding how they work allows engineers to design control systems, safety systems, automation and more.

Security systems

AND gates ensure multiple conditions must be met (e.g. correct PIN entered AND correct card present) before access is granted.

Industrial safety

Two-hand control presses require AND logic — both buttons must be pressed simultaneously before the press activates, keeping hands safe.

Car systems

Airbags deploy using complex combinational logic — multiple sensors (crash severity, speed, seatbelt status) feed AND/OR combinations.

Environmental control

NOT gates invert thermistor signals so heating switches on when temperature is LOW (not high), using NOT logic for active-low sensors.

Medical equipment

Ventilators use logic gates to decide when to deliver a breath — combinations of patient effort sensors and timing signals feed AND/OR gates.

Robotics

Obstacle avoidance — OR gate triggers braking if left sensor OR right sensor detects an obstacle. AND gates confirm both sensors clear before resuming.

Task 8 — Real-world design challenge

A fish and chip shop needs a warning light (Z) for their fryer. The warning light should come on if the oil is too hot (temperature sensor A = 1 when too hot) OR if the oil level has dropped below the minimum level (level sensor B = 0 when oil is low).

- Identify the gate(s) needed and explain your choice.
- Write the Boolean expression for Z.
- Draw the circuit diagram and complete the truth table. Verify using NoNANDSense.

A large hall has three temperature sensors A, B and C. The heating system (Z) should switch on if any TWO OR MORE of the three sensors fall below a preset level (logic 0 = too cold). Design and build the complete logic system.



NoNANDSense Simulator — open workspace

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.